

Bayes Watch: A Bayesian Network Approach to Probabilistic Reasoning in a Fictional Mystery-Solving Application

Tasnif Gaffar Pronoy, Md. Nazibul Islam Nabil, Nazifa Tahsin, Zinat Shaharin Mim, Jannat-A-Habib-Baishakhi

Department of Electrical and Computer Engineering

North South University, Dhaka, Bangladesh

Email: tasnif.pronoy@northsouth.edu, nazibul.nabil@northsouth.edu, nazifa.tahsin@northsouth.edu,

zinat.mim@northsouth.edu, jannat.baishakhi@northsouth.edu

Abstract—Reasoning under uncertainty is one of the central problems that artificial intelligence systems must solve, and Bayesian networks provide a compact, interpretable way to do so. This report presents *Bayes Watch: The Case of the Vanishing Sapphire*, an interactive mystery-solving application that models a fictional jewel-heist investigation as a Bayesian network. A single hidden variable, the identity of the guilty suspect, is connected to eight observable clue variables covering forced entry, motive, alibi, fingerprint evidence, and security footage. As a user supplies clue values through a web interface, the system performs exact probabilistic inference using the Variable Elimination algorithm to compute the posterior probability that each suspect is guilty, and updates a live bar chart and verdict banner accordingly. The application is implemented in Python using the `pgmpy` library for network construction and inference, `Streamlit` for the user interface, and `Graphviz` for visualizing the network structure. This report describes the problem formulation, the design of the network and its conditional probability tables, the mathematical basis of the inference procedure, the software architecture, and representative case studies that demonstrate how the posterior probabilities shift as evidence accumulates. We also discuss the limitations of the current design and directions for future extension.

Index Terms—Bayesian networks, probabilistic graphical models, Bayesian inference, Variable Elimination, Naive Bayes, decision support, Streamlit, pgmpy.

I. INTRODUCTION

UNCERTAINTY is unavoidable in almost every real-world reasoning task. A medical diagnosis system must decide which disease is most likely given a set of symptoms; a spam filter must decide whether an email is unwanted given a set of words; and, in the scenario explored in this project, a detective must decide which suspect is most likely guilty given a set of clues. In all of these cases, the underlying question has the same shape: given some observed evidence, what is the probability of each possible hidden cause? Bayesian networks are a natural tool for this class of problem because they represent the joint probability distribution over a set of random variables as a directed acyclic graph, factorized according to conditional independence relationships, which makes both the representation and the computation of posterior probabilities tractable [1], [2].

This project, developed for the course CSE440 (Artificial Intelligence), implements a Bayesian network-based decision support tool named *Bayes Watch*. The application places the user in the role of a detective investigating the disappearance of a fictional gemstone, the “Star of Midnight,” from a locked vault during a masquerade gala. Three suspects are available: an heiress with a motive rooted in an inheritance dispute, a butler with unsupervised access to the vault, and a professional jewel thief. As the user enters clues – for example, whether the vault was forced open, whether a suspect has an alibi, or whose fingerprints were found at the scene – the system recomputes, in real time, the posterior probability that each suspect committed the theft.

The purpose of the project is twofold. First, it is a functioning application that lets a user interactively explore how evidence changes belief, which is pedagogically useful for building intuition about Bayesian reasoning. Second, and more importantly for this report, every part of the system is deliberately exposed: the graph structure, the conditional probability tables (CPTs), and the exact inference algorithm are all visible to the user, so that the system is never a “black box.” This report documents the design and implementation of that system, explains the mathematics behind it, and evaluates its behavior through worked examples.

The remainder of this report is organized as follows. Section II reviews the background on Bayesian networks and their relationship to the Naive Bayes model. Section III describes the fictional case and the design of the network. Section IV formally specifies the network structure and conditional probability tables. Section V derives the inference procedure used to compute posterior probabilities. Section VI describes the software architecture and user interface. Section VII presents case studies and discusses the observed behavior of the system. Section VIII discusses limitations and future work, and Section IX concludes the report.

II. BACKGROUND AND RELATED WORK

A Bayesian network is a probabilistic graphical model that represents a set of random variables and their conditional dependencies through a directed acyclic graph (DAG) [1].

Each node in the graph corresponds to a random variable, and each edge represents a direct probabilistic influence of a parent variable on a child variable. The key property that makes Bayesian networks computationally useful is that the joint probability distribution over all n variables $X_1, \dots, X_n$ factorizes as

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Parents}(X_i)), \quad (1)$$

which is dramatically smaller than a full joint table when each variable has few parents [2]. Rather than storing an exponentially large joint distribution, the model only needs one conditional probability table (CPT) per node.

A particularly important special case, and the one most relevant to this project, occurs when a single hidden “class” variable C is the parent of every other (“feature”) variable $F_1, \dots, F_k$, with no edges between the feature variables themselves. This structure is exactly the graphical model underlying the *Naive Bayes classifier* [3], in which the joint distribution simplifies to

$$P(C, F_1, \dots, F_k) = P(C) \prod_{i=1}^k P(F_i \mid C). \quad (2)$$

The “naive” assumption is that, given the class, the features are conditionally independent of one another. Naive Bayes models are widely used in text classification and spam filtering precisely because this assumption, while simplifying, is often accurate enough in practice and yields fast, robust inference. The Bayes Watch network reuses this exact structure: the “class” variable is the identity of the guilty suspect, and the “features” are the eight observable clues.

Exact inference in a Bayesian network – computing $P(Q \mid E)$ for a query variable Q given evidence E – can in general be performed by the *Variable Elimination* algorithm, which exploits the factorization of Eq. (1) to avoid ever materializing the full joint distribution [2]. This project uses the open-source `pgmpy` library, which provides both a data structure for discrete Bayesian networks and an implementation of Variable Elimination [4]. The user interface is built with `Streamlit`, a Python framework for turning scripts into interactive web applications, and the network diagram is rendered with `Graphviz` [6].

III. CASE DESIGN AND PROBLEM FORMULATION

A. The Fictional Scenario

The application is framed as a locked-room mystery. On the night of the Ravenswood Masquerade Gala, a sapphire known as the Star of Midnight disappears from the manor’s private vault. Three characters are established as suspects, summarized in Table I.

Framing the target variable as a small, closed set of named suspects keeps the model easy to reason about while still being expressive enough to demonstrate genuine Bayesian updating: each suspect has a distinct profile that makes some clues more or less likely under that suspect’s guilt.

TABLE I: Suspects in the Bayes Watch Case

Suspect	Code	Why they are a suspect
Lady Odalys Voss (the Heiress)	A	Distant cousin in a long-running inheritance dispute; had vault access as guest of honor.
Jenkins (the Butler)	B	Thirty years of service; the only staff member holding a master key to the vault.
The Velvet Fox (the Jewel Thief)	C	A never-caught professional, reportedly active in the region this season.

B. Design Rationale: One Hidden Cause, Many Observed Effects

The central design decision is to model the problem as a *single hidden cause with multiple observable effects*, rather than as a densely connected network in which clues influence one another. This choice was made for three reasons. First, it mirrors how a detective actually reasons: the guilty party’s identity is the one true, unobserved fact that shapes everything else, while individual clues (an alibi, a fingerprint) are typically gathered independently of one another once the culprit is fixed. Second, it keeps every conditional probability table small (a table over the child’s states given only three possible parent values), which keeps the system easy to display in the “Peek at the Math” panel of the user interface. Third, as discussed in Section II, this structure is exactly the Naive-Bayes-shaped network, for which inference is particularly efficient and whose posterior can be understood with a single, easily explained formula.

A direct consequence of this structure is a set of conditional independence statements. For example, once the guilty party is known, the clue *ForcedEntry* is conditionally independent of the clue *AlibiA* – learning whether the vault was forced open provides no additional information about the heiress’s alibi, beyond what is already implied by knowing who is guilty. This property is what allows Variable Elimination to compute the exact posterior without building the full joint table over all nine variables.

IV. BAYESIAN NETWORK MODEL

A. Network Structure

The network contains one hidden variable and eight observable evidence variables, listed in Table II. The hidden variable, `GuiltyParty`, has three states, $\{A, B, C\}$, corresponding to the three suspects. Every evidence variable has `GuiltyParty` as its only parent; there are no edges between evidence variables. The resulting graph therefore has nine nodes and eight edges, all directed outward from the single root node `GuiltyParty`.

Note that no explicit `MotiveC` variable is defined for the professional thief: because a career criminal’s motive (the value of the jewel itself) is effectively constant, it was judged not to carry discriminative information and was intentionally left out of the model, a simplification we revisit in Section VIII.

TABLE II: Random Variables in the Bayes Watch Network

Variable	Domain	Interpretation
GuiltyParty (hidden)	$\{A, B, C\}$	Identity of the culprit
ForcedEntry	$\{\text{Yes, No}\}$	Was the vault forced open?
MotiveA	$\{\text{Yes, No}\}$	Strong motive found for the Heiress
MotiveB	$\{\text{Yes, No}\}$	Strong motive found for the Butler
AlibiA	$\{\text{Yes, No}\}$	Heiress has a verified alibi
AlibiB	$\{\text{Yes, No}\}$	Butler has a verified alibi
AlibiC	$\{\text{Yes, No}\}$	Jewel Thief has a verified alibi
Fingerprints	$\{\text{None, A, B, C}\}$	Whose fingerprints (if any) were found
SecurityFootage	$\{\text{Yes, No}\}$	Useful footage recovered



Fig. 1: Structure of the Bayes Watch Bayesian network. The hidden node `GuiltyParty` is the sole parent of all eight observable clue nodes; there are no edges between clue nodes, which gives the graph the same “one cause, many effects” shape as a Naive Bayes classifier.

TABLE III: Conditional Probability Table: $P(\text{ForcedEntry} \mid \text{GuiltyParty})$

	GP = A	GP = B	GP = C
ForcedEntry = Yes	0.20	0.10	0.85
ForcedEntry = No	0.80	0.90	0.15

Fig. 1 shows the network as rendered by the application’s Graphviz visualization, with the `GuiltyParty` node visually distinguished as the hidden root cause.

B. Conditional Probability Tables

Each edge in the network is backed by a conditional probability table (CPT) that encodes a piece of detective reasoning. As an illustration, Table III shows the CPT implemented for `ForcedEntry`. Because the professional thief does not possess a key, forced entry is made substantially more likely under `GuiltyParty = C` than under either of the two insiders, both of whom have legitimate means of entering the vault.

Table IV shows the CPT for `Fingerprints`, which has four outcomes instead of two. A professional thief is modeled as being the most careful about leaving forensic

TABLE IV: Conditional Probability Table: $P(\text{Fingerprints} \mid \text{GuiltyParty})$

	GP = A	GP = B	GP = C
None found	0.55	0.55	0.70
Matches A	0.35	0.05	0.05
Matches B	0.05	0.35	0.05
Matches C	0.05	0.05	0.20

evidence, hence the higher probability of no fingerprints being found under `GuiltyParty = C`; the two insiders are each somewhat more likely to leave their own print than someone else’s.

The remaining CPTs (motive, alibi, and security footage) follow the same design principle: each table is populated using qualitative reasoning about the suspects’ profiles (for example, an insider under suspicion is less likely to have a clean alibi than an unrelated party), and the resulting probabilities are always normalized so that each column sums to one, as required for a valid CPT. The full set of tables is available in the accompanying source code (`supports/mystery_solver.py`) and is displayed to the end user at run time through the application’s “Peek at the Math” panel, so that no probability used by the system is ever hidden from the investigator.

The prior distribution over `GuiltyParty` before any evidence is entered is uniform,

$$P(\text{GuiltyParty} = A) = P(\text{GuiltyParty} = B) = P(\text{GuiltyParty} = C) \quad (3)$$

reflecting the assumption that, absent any clues, all three suspects are equally likely to be guilty.

V. INFERENCE METHODOLOGY

A. Bayes’ Theorem for the Network

Let $E = \{e_1, \dots, e_m\}$ denote the subset of the eight clue variables for which the user has supplied a value (the remaining clues are left as “Unknown” and are simply absent from E). Because every evidence variable’s only parent is `GuiltyParty`, the joint distribution factorizes as in Eq. (2), and Bayes’ theorem gives the posterior over the guilty party directly:

$$P(\text{GuiltyParty} = g \mid E) = \frac{P(g) \prod_{e_i \in E} P(e_i \mid g)}{\sum_{g' \in \{A, B, C\}} P(g') \prod_{e_i \in E} P(e_i \mid g')} \quad (4)$$

The numerator is the (unnormalized) product of the prior and the likelihood of every observed clue under a candidate suspect g ; the denominator is a normalizing constant, often written Z or α^{-1} , that ensures the three posterior probabilities sum to one. Any clue variable that the user leaves as “Unknown” does not appear in E at all; because such a variable is a leaf node whose column of conditional probabilities sums to one, marginalizing it out of the joint distribution contributes a factor of exactly 1 and therefore has no effect on the posterior. This is precisely why the application can simply omit unanswered

clues from the evidence dictionary rather than needing to treat “Unknown” as a distinct observed state.

B. Variable Elimination

While Eq. (4) can be evaluated directly by hand for this particular “one cause, many effects” structure, the application uses `pgmpy`’s general-purpose *Variable Elimination* engine so that the same code would remain correct even for a more densely connected network. Variable Elimination computes $P(Q | E)$ for a query variable Q by:

- 1) instantiating every evidence variable in E to its observed value, which reduces each of its CPT columns to a single relevant row (a “factor”);
- 2) repeatedly picking a variable that is neither Q nor in E , multiplying together all factors that mention it, and *summing it out* (marginalizing over its domain) to produce a new, smaller factor;
- 3) multiplying together the remaining factors, all of which now depend only on Q , and normalizing the result so that it sums to one.

Because the algorithm eliminates variables one at a time and exploits the conditional independencies encoded by the graph, it never needs to construct the full joint distribution over all nine variables, which would otherwise require enumerating $3 \times 2^6 \times 4 = 768$ joint outcomes. This computational saving becomes more significant as networks grow larger and more densely connected than the one used in this project.

VI. SYSTEM IMPLEMENTATION

A. Software Architecture

The codebase is organized to separate the probabilistic model from the presentation layer, as shown in Fig. 2. The file `main.py` is responsible only for the Streamlit user interface: laying out the page, collecting clue values from sidebar widgets, invoking inference, and displaying results. All of the Bayesian network logic – constructing the graph, defining every CPT, and rendering the Graphviz diagram – is implemented in `supports/mystery_solver.py` and exposed through three functions: `build_bayesian_network()`, which returns a `pgmpy DiscreteBayesianNetwork` object; `create_graphviz_plot()`, which builds the visual graph shown in Fig. 1; and `cpd_to_dataframe()`, which converts a raw CPT into a `pandas DataFrame` for display. This separation means the underlying inference logic never has to change even if the fictional theme, suspect names, or wording were replaced with a different case.

At start-up, `main.py` calls `build_bayesian_network()` to obtain the model and wraps it in a `pgmpy VariableElimination` object, which is reused for every query issued during the session. Graphviz rendering is treated as an optional dependency: if the `graphviz` package or its underlying system binaries are not available, the application degrades gracefully and continues to operate without the diagram, since the diagram is a visualization aid and not a requirement for inference.

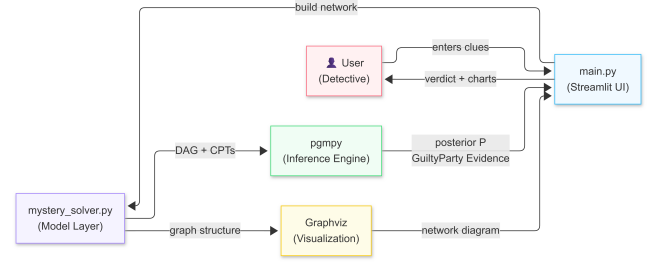


Fig. 2: Software architecture of Bayes Watch. The Streamlit interface layer collects user input and displays results; the model layer builds the Bayesian network and its CPTs; `pgmpy`’s Variable Elimination engine performs exact inference; Graphviz renders the network diagram.

B. User Interface

The interface is split into a sidebar for evidence entry and a main panel for results, shown together in Fig. 3. The sidebar groups the eight clue widgets under three headings – *Scene of the Crime* (forced entry), *Motive*, *Alibis*, and *Forensics* (fingerprints, security footage) – each rendered as a drop-down with the options “Yes,” “No,” and “Unknown.” Selecting “Unknown” maps internally to `None` and is filtered out of the evidence dictionary before the query is issued, consistent with the marginalization argument in Section V. A “Reset All Clues” button returns every widget to “Unknown” without reloading the page, implemented through Streamlit’s session-state mechanism.

When the user presses “Solve the Mystery,” the main panel displays the evidence considered, a table and bar chart of the posterior probability of guilt for each suspect, and a verdict banner. The verdict banner applies a simple, transparent rule on top of the computed posterior: if the leading suspect’s probability is at least 60%, the case is declared “closed”; if the gap between the top two suspects is below 8 percentage points, the result is reported as “too close to call”; otherwise, the leading suspect is reported with an explicit caveat that the case is not yet airtight. Before any clue is entered, the same panel instead shows the prior distribution of Eq. (3), so the page is informative even before the user interacts with it. An expandable “Peek at the Math” panel additionally prints every raw CPT used by the model, directly addressing the goal of keeping the reasoning process transparent.

VII. RESULTS AND CASE STUDIES

To evaluate the qualitative behavior of the model, we walk through a representative case study using the CPTs described in Section IV and Eq. (4).

A. Case Study: A Clean Insider Suspect

Suppose the investigator enters two clues: the vault was *not* forced open (`ForcedEntry = No`), and the recovered fingerprints match the butler (`Fingerprints = B`). Applying Eq. (4) with the prior of Eq. (3) and the CPT values in

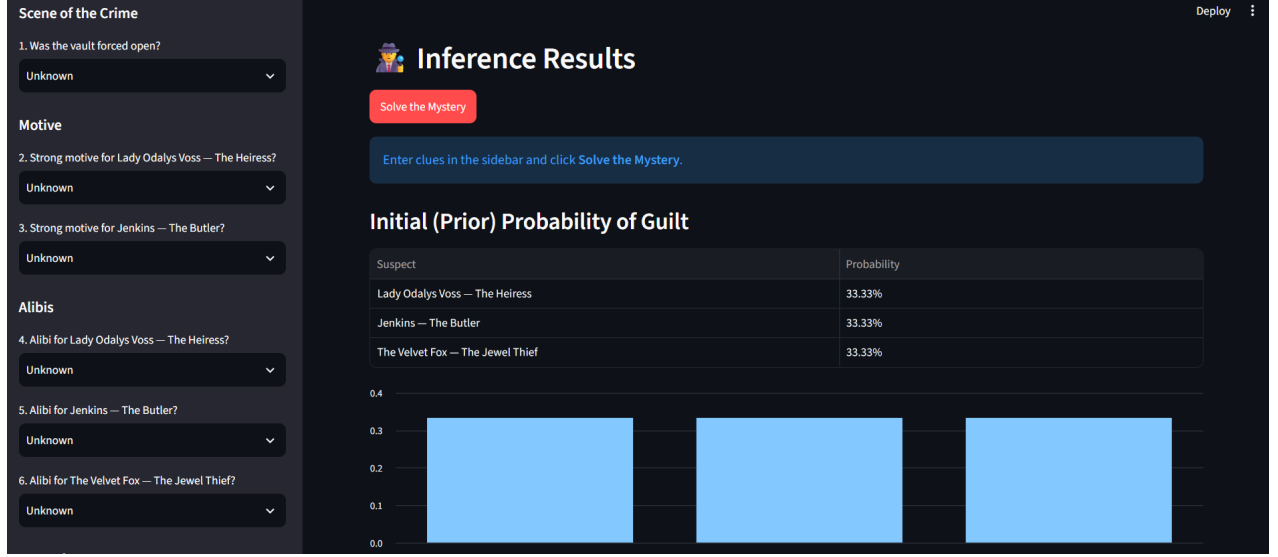


Fig. 3: The Bayes Watch interface: the sidebar (left) for entering clues alongside the main results panel (right), which shows the evidence considered, the posterior probability table, the bar chart of $P(\text{GuiltyParty} | E)$, and the verdict banner for the evidence used in Case Study 1 (Section VII).

TABLE V: Posterior Probabilities Before and After Evidence (Case Study 1)

	Heiress (A)	Butler (B)	Thief (C)
Prior	33.3%	33.3%	33.3%
Posterior	11.0%	86.9%	2.1%

Tables III and IV, the unnormalized posterior for each suspect is

$$\tilde{P}(A) = \frac{1}{3}(0.80)(0.05) = 0.01333, \quad (5)$$

$$\tilde{P}(B) = \frac{1}{3}(0.90)(0.35) = 0.10500, \quad (6)$$

$$\tilde{P}(C) = \frac{1}{3}(0.15)(0.05) = 0.00250. \quad (7)$$

The normalizing constant is $Z = 0.01333 + 0.10500 + 0.00250 = 0.12083$, which yields the posterior distribution

$$P(A | E) \approx 11.0\%, \quad P(B | E) \approx 86.9\%, \quad P(C | E) \approx 2.1\%. \quad (8)$$

This matches the intuitive reading of the evidence: an unforced entry is consistent with someone who already had a key, and a direct fingerprint match makes the butler’s guilt overwhelmingly likely. Because $P(B | E)$ exceeds the 60% threshold, the application’s verdict banner would declare “Case Closed” in favor of Jenkins the Butler. Table V summarizes the shift from the uniform prior to this posterior.

B. Case Study: Ambiguous Evidence

As a contrasting example, if the only clue entered is that the vault was forced open ($\text{ForcedEntry} = \text{Yes}$), the unnormalized posterior is $\tilde{P}(A) = \frac{1}{3}(0.20) = 0.0667$, $\tilde{P}(B) = \frac{1}{3}(0.10) = 0.0333$, and $\tilde{P}(C) = \frac{1}{3}(0.85) = 0.2833$, giving $Z = 0.3833$ and a posterior of $P(A | E) \approx 17.4\%$, $P(B | E) \approx 8.7\%$, and $P(C | E) \approx 73.9\%$. Although

the Velvet Fox becomes the leading suspect, this single clue alone is not, in general, decisive; when two or more clues offer conflicting signals about different suspects, the resulting margin between the top two candidates can fall below the 8-percentage-point threshold used by the verdict banner, in which case the system correctly reports the case as “too close to call” rather than overstating its confidence. This behavior illustrates a broader property of Bayesian updating: each additional independent clue multiplies into the posterior, so belief becomes more concentrated only as genuinely corroborating evidence accumulates, and can just as easily remain spread across multiple suspects when the evidence is weak or contradictory.

C. Discussion of Results

These case studies demonstrate two properties that are important for a decision-support tool. First, the system’s confidence tracks the strength of the evidence in an interpretable way: a single strong, specific clue (a fingerprint match) can move the posterior far more than a weaker, more general clue (forced entry alone), because the CPTs were designed so that the fingerprint variable is more discriminative between suspects. Second, because the verdict banner is derived directly from the same posterior distribution shown to the user, there is no discrepancy between the numeric result and the natural-language summary – both come from the same underlying computation, which is central to the project’s goal of avoiding a “black box” presentation of AI-driven conclusions.

VIII. DISCUSSION, LIMITATIONS, AND FUTURE WORK

The current design has several limitations that are worth stating explicitly. First, the network assumes conditional independence between all eight clues given the guilty party;

this is the same simplifying assumption used by Naive Bayes classifiers, and while it keeps the CPTs small and the reasoning transparent, it cannot capture situations in which two clues are correlated for reasons unrelated to guilt (for example, a broken alarm might make both “forced entry” and “no useful footage” more likely regardless of who is guilty). Second, as noted in Section IV, no motive variable exists for the professional thief, which means motive evidence can only ever discriminate between the two insider suspects. Third, all conditional probabilities were hand-specified based on narrative reasoning about the fictional case rather than learned from data, which is appropriate for a demonstration system but would not generalize to a real investigative setting without a process for eliciting or learning these probabilities from historical case data.

Several extensions follow naturally from these limitations and were also identified during development: (i) adding a suspect-specific motive clue for the jewel thief so that all three suspects are symmetric with respect to motive evidence; (ii) allowing the user to choose between multiple independent mystery scenarios, each with its own network, to demonstrate that the same inference engine generalizes across cases; (iii) adding a sensitivity-analysis view that reports, for each unanswered clue, how much it would be expected to shift the posterior if answered, which would help a user decide what evidence to look for next; and (iv) persisting a log of past investigations so that multiple sessions can be compared. These extensions would not require any change to the inference method described in Section V; they only require adding nodes, CPTs, or bookkeeping around the existing model, which reflects a broader strength of the Bayesian network formalism – structure and parameters can be extended incrementally without redesigning the reasoning procedure itself.

IX. CONCLUSION

This report presented Bayes Watch, an interactive application that uses a Bayesian network to solve a fictional jewel-heist mystery. The system models the identity of the guilty suspect as a single hidden variable governing eight observable clues, giving the network the same “one cause, many effects” structure used by the Naive Bayes classifier. User-supplied evidence is combined with hand-specified conditional probability tables through Bayes’ theorem, and the exact posterior probability of guilt for each suspect is computed using the Variable Elimination algorithm provided by the `pgmpy` library. A Streamlit-based interface presents this reasoning process transparently: the network diagram, every conditional probability table, and the resulting posterior distribution are all visible to the user rather than hidden behind a single opaque prediction. Worked case studies showed that the system’s confidence scales sensibly with the specificity of the evidence provided, correctly favoring a decisive verdict when the evidence strongly implicates one suspect and correctly withholding a confident verdict when the evidence is weak or ambiguous. Beyond its role as a course project, the system serves as a

concrete, explorable illustration of how probabilistic graphical models turn scattered pieces of evidence into a principled, quantitative belief – the same underlying mechanism used far more broadly in medical diagnosis, spam filtering, and other AI systems that must reason under uncertainty.

REFERENCES

- [1] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. San Mateo, CA, USA: Morgan Kaufmann, 1988.
- [2] D. Koller and N. Friedman, *Probabilistic Graphical Models: Principles and Techniques*. Cambridge, MA, USA: MIT Press, 2009.
- [3] I. Rish, “An empirical study of the naive Bayes classifier,” in *Proc. IJCAI 2001 Workshop on Empirical Methods in Artificial Intelligence*, 2001, pp. 41–46.
- [4] A. Ankan and A. Panda, “pgmpy: Probabilistic graphical models using Python,” in *Proc. 14th Python in Science Conf. (SciPy)*, 2015, pp. 6–11.
- [5] Streamlit Inc., “Streamlit documentation,” 2024. [Online]. Available: <https://docs.streamlit.io>
- [6] J. Ellson, E. Gansner, L. Koutsofios, S. C. North, and G. Woodhull, “Graphviz – open source graph drawing tools,” in *Graph Drawing*, Lecture Notes in Computer Science, vol. 2265, Springer, 2001, pp. 483–484.
- [7] Bayes Watch project repository, “CSE440-Project,” GitHub, 2026. [Online]. Available: <https://github.com/Nabil642/CSE440-Project>
- [8] Anthropic, “Claude (Sonnet),” large language model used as a writing and editing aid in the preparation of this report, 2026. [Online]. Available: <https://www.anthropic.com/claude>